

AI for Science Applied to Financial Advice: An Agent-Governed, Advisor-in-the-Loop Architecture for Household Wealth Strategy Discovery

Alejandro Reynoso

Chief Scientist, DEFI Capital

June 10, 2026

Abstract

This paper develops an agent-governed architecture for AI-assisted household wealth planning inspired by the *AI for Science* (AI4S) paradigm. Rather than treating a large language model as a financial advisor or portfolio oracle, the system places generative AI inside a closed-loop professional workflow in which candidate strategies are proposed, screened for fiduciary suitability, simulated across stochastic market paths, stress-tested under adverse scenarios, reviewed by a Certified Financial Planner persona, and rejected, revised, or retained under explicit governance criteria. The architecture is implemented in a companion Colab notebook centered on Sam and Taylor Reeves, a dual-income household with a \$1.33 million portfolio, a \$60,000 annual savings capacity, moderate risk tolerance, and three ranked goals: retirement security, college funding, and a vacation-home down payment.

The paper specifies the workflow, software architecture, computational core, agent decision protocol, suitability safeguards, LLM interfaces, stress scenarios, behavioral review layer, and audit artifacts required for an auditable AI-assisted wealth-planning discovery prototype. The central methodological distinction is between *strategy generation* and *fiduciary authorization*. The strategy-generation role may propose allocations, account priorities, and implementation ideas, but it is not the role that screens suitability, computes outcomes, performs CFP review, or marks its own candidate as accepted. The paper also distinguishes portfolio weights from professional planning judgments such as risk capacity, liquidity reserves, funded status, tax location, account eligibility, and behavioral durability. The system’s goal-attainment estimates are produced by a fixed-seed Monte Carlo engine using a synthetic market universe and correlation-aware portfolio statistics. The implementation also reports product fee drag, tax-efficiency scoring, and risk flags as audit metrics around those simulations. The final continuation or stopping decision is governed by a `WealthEvaluationAgent`, and no strategy can be accepted without review by an `AdvisorReviewAgent` simulating Robert Okafor, CFP.

The contribution is methodological. The system is not investment advice, does not recommend any security, and does not replace a licensed fiduciary advisor. Instead, it provides a blueprint for using AI in financial planning under controlled professional conditions: explicit client facts, ranked goals, synthetic capital-market assumptions, suitability gates, stochastic simulation, stress tests, behavioral warnings, advisor-in-the-loop review, decision logs, and investment-committee-style approval discipline. The resulting architecture demonstrates how AI4S can be adapted from scientific discovery to professional wealth planning. Its value lies not in automating judgment away, but in making financial advice exploration more systematic, transparent, auditable, and accountable.

Keywords: artificial intelligence for science; financial advice; wealth planning; Monte Carlo simulation; goal-based investing; agentic AI; fiduciary suitability; advisor-in-the-loop; behavioral risk; investment committee governance; model risk management.

1. Introduction

Wealth planning is a household balance-sheet problem under uncertainty, not merely an investment-selection problem. A household arrives with income, assets, debts, account structures, tax exposure, liquidity needs, risk tolerance, risk capacity, behavioral tendencies, and goals that are not equally important. The professional task is to raise the likelihood of satisfying the household’s most important liabilities while preserving feasibility,

liquidity, tax efficiency, and behavioral durability. This framing maps naturally onto the AI4S paradigm: a system proposes candidate solutions, tests them against structured evidence, learns from failure, and refines its search direction accordingly. The market replaces the laboratory. Monte Carlo paths replace experiments. Goal-attainment probability replaces reaction yield.

The critical difference from a standard optimizer is the word *ranked*. Most portfolio optimizers treat investor welfare as a single scalar objective: maximize expected utility, maximize expected wealth, or maximize return for a given level of variance. Real clients do not experience their financial lives that way. Retirement security is not the same kind of goal as a vacation home. One is non-negotiable and carries decades of income dependency; the other is aspirational and can be deferred, reduced, or abandoned. A fiduciary plan should therefore distinguish goal priority, time horizon, funded status, risk capacity, and willingness to bear losses. A system that treats all goals symmetrically may produce an allocation that looks efficient on paper but fails the client in the scenario that matters most. Getting the hierarchy wrong is therefore not a small modeling error. It is a form of misadvice.

The architecture developed here applies the AI4S grammar to financial advice. A synthetic market universe defines asset classes, return and volatility assumptions, key correlations, product menus, fees, account types, regulatory constraints, inflation, tax assumptions, and safe-withdrawal parameters. A client profile defines Sam and Taylor Reeves, a married household with combined gross income of \$310,000, estimated after-tax income of \$215,000, annual savings capacity of \$60,000, and a total portfolio of approximately \$1.33 million. Their current portfolio has several defects: concentration in three technology stocks, cash-equivalent drag that must be reconciled with emergency-reserve needs, an unrebated 401(k), limited inflation protection, underfunded 529 accounts, and paused Roth IRA contributions. Their goals are ranked: retirement security at age sixty-two, college funding for two children, and a vacation-home down payment.

The notebook system has three agentic components. The `WealthStrategyLoop` runs one complete research cycle: it generates candidate investment strategies via Claude, passes them through a suitability gate that checks fiduciary compliance, runs 1,000 Monte Carlo simulations across the fifteen-year primary horizon, runs 500-path simulations for each goal-specific horizon, estimates goal-attainment probabilities for each ranked goal, stress-tests the best strategy under a market crash, inflation spike, and sequence-of-returns scenario, tracks a short-horizon shadow portfolio proxy, obtains strategic feedback, and generates search seeds for the next cycle. The `AdvisorReviewAgent` simulates Robert Okafor, CFP, a senior wealth advisor who reviews the selected AI-generated recommendation in each cycle for fiduciary suitability, goal hierarchy, behavioral realism, fee justification, and client communication. The `WealthEvaluationAgent` governs the full multi-round process, accumulating failure context and deciding whether the system should continue, stop successfully, or halt after the maximum number of rounds.

The design is deliberately conservative. It does not ask the LLM to compute portfolio outcomes directly. It does not allow unsuitable strategies to advance simply because they appear to improve returns. It does not allow a mathematical score to override behavioral feasibility or fiduciary judgment. Instead, it separates creativity, suitability screening, stochastic simulation, stress testing, behavioral review, advisor judgment, and final governance into distinct modules. This separation is the central methodological contribution of the paper.

The implementation is contained in a companion Colab notebook. The executable workflow is organized into nine code cells accompanied by contextual, architecture, reference, and conclusion Markdown. The cells cover setup and the Claude client, synthetic market universe, client profile, Monte Carlo engine, `WealthStrategyLoop`, `AdvisorReviewAgent`, `WealthEvaluationAgent`, orchestrator, and final wealth-plan dashboard. The notebook is compact enough for classroom demonstration but rich enough to illustrate the institutional logic of governed AI in financial advice.

The paper is organized as follows. Section 2 presents the end-to-end workflow. Section 3 describes the software architecture and the separation between the inner wealth-strategy loop and the outer evaluation agent. Section 4 specifies the computational core used for portfolio statistics, Monte Carlo simulation,

goal-attainment scoring, fee drag, tax-efficiency scoring, stress testing, and strategy selection. Section 5 formalizes the evaluation-agent protocol. Section 6 describes suitability, behavioral, numerical, and governance safeguards. Section 7 explains the LLM integration. Section 8 presents the stress-testing scenarios. Section 9 states the investment-committee review standard. Section 10 discusses contributions, limitations, and future research. The appendix documents the companion notebook and includes the workflow diagram.

The intended contribution is methodological. The paper does not claim that a synthetic market universe can substitute for actual capital-market research, nor that an LLM can replace a fiduciary advisor. Instead, it demonstrates how AI can be placed inside a disciplined professional workflow: one that records client facts, respects ranked goals, screens recommendations for suitability, simulates stochastic outcomes, stress-tests fragility, surfaces behavioral risks, preserves advisor judgment, and maintains an auditable decision trail.

2. Workflow Overview

The workflow is a closed-loop wealth-strategy discovery procedure rather than a one-shot asset-allocation recommendation. Each cycle begins with structured client facts and a synthetic market universe, generates candidate strategies, screens them through a suitability gate, simulates their outcomes through Monte Carlo paths, evaluates each ranked goal separately, stress-tests the strongest strategy, runs a shadow-portfolio proxy, obtains strategic feedback, generates next-cycle seeds, and sends the result to a fiduciary advisor review layer. The outer evaluation agent then decides whether to stop or continue.

Table 1 summarizes the ten-stage workflow implemented in the notebook.

Table 1: Ten-stage AI4S financial-advice workflow.

Stage	Description
1. Synthetic Market Universe	Encodes asset classes, return and volatility assumptions, key correlations, product menu, fees, account types, regulations, inflation, and tax assumptions.
2. Client Profile	Defines Sam and Taylor Reeves, including income, portfolio, current allocation, risk tolerance, liquidity needs, known portfolio issues, and ranked goals.
3. Baseline Simulation	Computes current-plan outcomes using the existing allocation, a \$30,000 annual contribution, and a product mix of 60 percent active mutual funds and 40 percent index ETFs.
4. Strategy Generation	Claude proposes exactly five investment strategies with allocation, product mix, annual contribution, account priorities, rationale, target goal, and risk label.
5. Suitability Gate	Claude acts as compliance officer and classifies each strategy as <code>SUITABLE</code> , <code>BORDERLINE</code> , or <code>UNSUITABLE</code> .
6. Monte Carlo Simulation	Seeded code functions simulate terminal wealth and goal attainment across stochastic market paths.
7. Goal-Weighted Evaluation	The system computes separate probabilities for retirement, college, and vacation-home goals and combines them using priority weights.
8. Stress and Shadow Tests	The best strategy is tested under market crash, inflation spike, and sequence-risk scenarios, plus a 30-day shadow-portfolio proxy.
9. Advisor Review	Robert Okafor, CFP, reviews the AI-generated strategy for suitability, hierarchy, behavioral realism, fees, modifications, and re-plan triggers.
10. Evaluation Decision	<code>WealthEvaluationAgent</code> decides whether to continue, stop successfully, or halt after maximum rounds.

The workflow contains two nested loops. The inner loop is implemented by `WealthStrategyLoop`. This loop performs one complete wealth-planning research cycle: strategy generation, suitability screening, simulation, stress testing, shadow tracking, feedback, and next-cycle seed generation. The outer loop is

implemented by `WealthEvaluationAgent`. This agent governs the full process across up to three research cycles, accumulates failure context, stores rejected strategy names, records CFP verdicts, and determines whether further search is justified.

The critical control point is the suitability-and-advisor gate. A strategy is not accepted solely because it improves expected terminal wealth or produces a higher goal score. It must avoid an `UNSUITABLE` verdict, remain plausible for a moderate-risk household, respect the liquidity requirement in the suitability and advisor prompts, remain consistent with fiduciary constraints, avoid unjustified product complexity, improve goal attainment relative to baseline, survive stress testing, and receive advisor approval or approval with modifications. This architecture converts AI-assisted financial advice from a plausible portfolio narrative into a governed discovery process.

The workflow also imposes a hierarchy of evidence. A generated strategy begins as a hypothesis. It gains evidentiary status only after it is expressed as a structured allocation, product mix, contribution level, and account priority. It gains additional status after suitability screening. It gains further credibility if Monte Carlo simulation produces acceptable goal probabilities. It becomes more robust if it survives market-crash, inflation, and sequence-risk stress tests. It becomes professionally plausible only after CFP review. Finally, it becomes a candidate recommendation only if the evaluation agent records a `STOP_SUCCESS` decision. This hierarchy is central to the AI4S mapping: the model proposes, but governance decides.

3. Software Architecture

The implementation is organized around four layers: a synthetic market and regulatory environment, a fixed-seed Monte Carlo and goal-attainment engine, an inner wealth-strategy loop, and an outer governance agent. This separation is not merely a software preference. It is the primary mechanism by which the notebook preserves professional accountability. Each layer has a specific responsibility and a bounded authority.

3.1. Synthetic Market Universe

The `MARKET_UNIVERSE` object defines the investment environment. It contains annualized return and volatility assumptions for U.S. large-cap equity, U.S. small-cap equity, international developed equity, emerging-market equity, aggregate bonds, TIPS, high-yield bonds, real estate investment trusts, commodities, and cash equivalents. It also contains simplified key correlations, investment product definitions, account types, regulatory rules, inflation assumptions, tax assumptions, and a safe-withdrawal rate.

The product menu includes index ETFs, active mutual funds, target-date funds, variable annuities, individual bonds, and private equity. Each product carries fee, liquidity, and tax-efficiency characteristics. The account-type rules include 401(k), Roth IRA, traditional IRA, 529 plan, taxable account, and HSA. The regulatory framework includes fiduciary standard, suitability rules, fee-disclosure requirements, concentration limits, illiquid-asset limits, leverage restrictions, churning constraints, and accredited-investor criteria.

The purpose of this synthetic market universe is not to reproduce current capital-market assumptions or regulatory law. Its methodological role is to create a controlled environment in which AI-generated strategies can be tested against explicit assumptions. The generation prompt constrains the allowed asset-class keys, product keys, allocation sums, product-mix sums, maximum annual contribution of \$60,000, single-asset-class exposure for a moderate-risk client, leverage, and illiquid-product exposure. The account-type contribution limits are recorded in `MARKET_UNIVERSE`, but the prototype does not yet perform deterministic account-level contribution-limit enforcement.

3.2. Client Profile and Ranked Goals

The `CLIENT_PROFILE` object defines the household case. Sam and Taylor Reeves are ages forty-seven and forty-four. Their combined gross income is \$310,000, estimated after-tax income is \$215,000, annual savings capacity is \$60,000, and total portfolio value is approximately \$1.33 million. Their current allocation is heavily tilted toward U.S. large-cap equity and cash equivalents, with limited diversification into small cap, international equity, aggregate bonds, and no explicit TIPS allocation.

The case profile also records known portfolio issues. The taxable account is concentrated in three technology stocks representing more than forty percent of that account. Cash savings are earning only 0.5 percent. Sam's 401(k) has not been rebalanced in three years. The portfolio has no explicit inflation hedge. The 529 accounts are underfunded. Roth IRA contribution streams were halted two years earlier.

The most important feature of the profile is the ranked goal structure. Retirement security is the top priority, with a \$3.6 million target in fifteen years and low flexibility. College funding is second, with a \$640,000 target in eight years and medium flexibility. The vacation-home down payment is third, with a \$195,000 target in seven years and high flexibility. The notebook translates this hierarchy into goal weights of 60 percent, 28 percent, and 12 percent, respectively. This weighting embeds the client value system directly into the objective function.

A senior-advisor interpretation of the case is that Sam and Taylor have both a funded-status problem and an implementation-quality problem. The household has meaningful savings capacity and a long enough retirement horizon to accept some growth risk, but its current portfolio also contains uncompensated concentration risk, weak inflation hedging, underfunded education accounts, and unresolved account-priority questions. The \$85,000 cash balance is not simply idle capital: the profile records a \$90,000 liquidity need, so any recommendation should improve yield and deploy only true surplus without impairing the emergency reserve. This is the difference between portfolio optimization and professional advice. The optimizer sees weights; the advisor sees funded ratios, liquidity reserves, tax location, contribution eligibility, behavioral capacity, and goal triage.

3.3. Monte Carlo and Goal-Attainment Engine

The fixed-seed simulation engine is implemented by `portfolio_stats()`, `monte_carlo()`, and `simulate_strategy()`. The portfolio-statistics function computes expected return, volatility, and Sharpe ratio from the allocation and correlation assumptions. The Monte Carlo function generates lognormal return paths and terminal wealth distributions. The strategy simulator computes median terminal wealth, 10th percentile terminal wealth, 90th percentile terminal wealth, simulated maximum drawdown, goal probabilities, weighted goal-attainment score, fee drag, tax-efficiency score, and risk flags. It uses 1,000 paths for the primary fifteen-year terminal distribution and 500 paths for each goal-specific probability estimate.

This layer is the computational center of the notebook. The LLM may generate a strategy, but it does not directly assign the goal score used by the system. The outcome estimate is calculated by code-controlled simulation rather than LLM assertion. This separation between financial-language generation and quantitative evaluation is essential for auditability. A reviewer can inspect which allocation was used, which product mix created the fee drag, how goal probabilities were computed, and why risk flags were generated.

3.4. WealthStrategyLoop

`WealthStrategyLoop` executes one research cycle. It is responsible for generating strategies, running the suitability gate, simulating all non-unsuitable strategies, selecting the best admissible strategy by goal-attainment score, stress-testing that strategy, running a 30-day shadow-portfolio proxy, obtaining strategic feedback, and generating next-cycle seeds. The output is stored in a `WealthCycleResult` dataclass.

The dataclass contains cycle number, generated strategies, suitability verdicts, simulation results, best strategy, best simulation, baseline score, stress results, fragility status, shadow return, feedback verdict, and next seeds. This object functions as an audit record. It preserves both positive and negative evidence and prevents the process from relying on informal memory or hidden state.

3.5. AdvisorReviewAgent

AdvisorReviewAgent simulates Robert Okafor, CFP, a senior wealth advisor with twenty-two years of practice. His role is not to generate new strategies, but to review the AI-generated recommendation. He evaluates fiduciary suitability, goal hierarchy, behavioral realism, fee justification, client communication, required modifications, behavioral warnings, and re-plan triggers. His verdict may be APPROVE, APPROVE_WITH_MODS, or REJECT.

This layer represents professional accountability. A strategy that passes simulation may still be unsuitable if it is behaviorally unrealistic, too complex, too expensive, misaligned with goal priority, or vulnerable to client panic during drawdowns. Robert Okafor is therefore not a cosmetic reviewer. In the notebook, the evaluation agent sends every selected cycle winner to this advisor-review layer before making a continuation or stopping decision; the heuristic recommends STOP_SUCCESS only when the advisor verdict is APPROVE or APPROVE_WITH_MODS and the numerical gates also clear.

3.6. WealthEvaluationAgent

WealthEvaluationAgent is the outer governor. It controls the maximum number of rounds, stores round results, stores advisor reviews, accumulates failure context, records rejected strategy names, and maintains a decision log. It sets $\text{MAX_ROUNDS} = 3$, $\text{MIN_GOAL_SCORE} = 0.65$, and $\text{MIN_IMPROVEMENT} = 0.05$. The heuristic stop condition requires advisor approval or approval with modifications, a goal-attainment score of at least 65 percent, improvement of at least five percentage points over baseline, and non-fragility under stress testing.

The outer governor prevents uncontrolled search. Without this layer, a generative system could continue producing strategies until it finds one that looks favorable or rhetorically compelling. The evaluation agent imposes round limits, goal-score thresholds, improvement thresholds, fragility controls, advisor-review constraints, and failure-context memory. It converts AI-assisted wealth planning into a bounded professional workflow.

4. Mathematical and Computational Core

The computational core defines the quantities the system is allowed to compute and compare. This formal layer is essential because LLMs are fluent in financial language but must not be permitted to invent terminal wealth, portfolio risk, Sharpe ratios, or goal-attainment probabilities after observing results. The notebook therefore specifies portfolio statistics, Monte Carlo simulation, goal-attainment scoring, fee drag, tax efficiency, risk flags, and stress fragility through explicit code.

4.1. Portfolio Expected Return and Volatility

Let $\mathcal{A}$ denote the set of available asset classes. A candidate strategy h specifies an allocation vector $\mathbf{w}^{(h)} = (w_1, \dots, w_n)$ over those asset classes, with $\sum_i w_i = 1$. Each asset class i has expected annual return μ_i and annual volatility σ_i . The portfolio expected return is

$$\mu_p^{(h)} = \sum_{i \in \mathcal{A}} w_i^{(h)} \mu_i. \quad (1)$$

The portfolio variance incorporates both individual volatilities and selected pairwise correlations:

$$\sigma_p^{2(h)} = \sum_i \left(w_i^{(h)}\right)^2 \sigma_i^2 + 2 \sum_{i < j} w_i^{(h)} w_j^{(h)} \rho_{ij} \sigma_i \sigma_j. \quad (2)$$

The portfolio volatility is $\sigma_p^{(h)} = \sqrt{\sigma_p^{2(h)}}$. The notebook also computes a Sharpe ratio using a synthetic risk-free rate $r_f = 4.5\%$:

$$\text{Sharpe}^{(h)} = \frac{\mu_p^{(h)} - r_f}{\sigma_p^{(h)}}. \quad (3)$$

These statistics are not presented as market forecasts. They are inputs to a controlled simulation environment that allows strategies to be compared under the same assumptions. In the current notebook, allocations are normalized before simulation, the correlation structure uses a sparse set of key pairs, and unstated cross-asset correlations are effectively ignored. That is acceptable for a transparent prototype, but a production-grade planning engine would need a validated positive-semidefinite covariance matrix and documented capital-market assumption governance. The Sharpe ratio is computed by `portfolio_stats()` before the later reporting adjustment for product fee drag.

4.2. Fee Drag and Reported Expected Return

Each strategy also specifies a product mix over available vehicles, such as index ETFs, active mutual funds, target-date funds, annuities, individual bonds, or private equity. Let $q_j^{(h)}$ denote the product weight and f_j the expense ratio for product j . The annual fee drag is

$$F^{(h)} = \sum_j q_j^{(h)} f_j. \quad (4)$$

The prototype records this drag and subtracts it from the reported expected portfolio return stored in the simulation result:

$$\tilde{\mu}_p^{(h)} = \mu_p^{(h)} - F^{(h)}. \quad (5)$$

This is conceptually important. A strategy with attractive gross expected returns can be unsuitable if high product costs erode the value delivered to the client. In the notebook implementation, however, the stochastic terminal wealth and goal-probability paths are generated by `monte_carlo()` from allocation-level return assumptions rather than from the fee-adjusted reported return. Fee drag therefore functions as a reported metric, dashboard field, and advisor-review input in the prototype, while a production version should feed net-of-fee assumptions directly into the path generator.

4.3. Monte Carlo Wealth Paths

For a strategy h with annual contribution $C^{(h)}$, the notebook generates lognormal return paths over the planning horizon. Let $V_{s,t}^{(h)}$ denote wealth in simulation path s at year t . The simplified accumulation process is

$$V_{s,t}^{(h)} = V_{s,t-1}^{(h)} R_{s,t}^{(h)} + C^{(h)}, \quad (6)$$

where $R_{s,t}^{(h)}$ is a simulated gross annual return factor drawn from a lognormal distribution parameterized by the allocation-level portfolio expected return and volatility. The notebook uses 1,000 simulated paths for primary strategy evaluation and a fifteen-year primary horizon. From the terminal distribution it reports median terminal wealth, 10th percentile terminal wealth, and 90th percentile terminal wealth.

The simulator also estimates drawdown using the median wealth path. Let $\bar{V}_t^{(h)}$ denote the median path at year t . The maximum drawdown proxy is

$$MDD^{(h)} = \min_t \left(\frac{\bar{V}_t^{(h)}}{\max_{u \leq t} \bar{V}_u^{(h)}} - 1 \right). \quad (7)$$

This drawdown estimate is used to flag strategies inconsistent with the client's stated drawdown tolerance. It is deliberately described as a proxy: the notebook measures drawdown on the median annual wealth path, not on every simulated path, and it does not model intra-year losses. A practitioner would treat this output as a screening signal rather than as a full behavioral-risk or capital-at-risk measure.

4.4. Goal-Attainment Probabilities

Each goal g has a target amount T_g , a horizon H_g , a rank r_g , and a flexibility label. The notebook inflates the target using the synthetic inflation rate ι :

$$\tilde{T}_g = T_g(1 + \iota)^{H_g}. \quad (8)$$

For each goal, the simulator runs 500 stochastic paths to the goal-specific horizon and estimates the probability of meeting the inflated target:

$$P_g^{(h)} = \frac{1}{N} \sum_{s=1}^N \mathbf{1} \left[V_{s,H_g}^{(h)} \geq \tilde{T}_g \right]. \quad (9)$$

This is the central move from portfolio optimization to goal-based advice. The strategy is not evaluated only by terminal wealth at fifteen years. It is evaluated by its probability of satisfying each client goal at the time that goal matters. The implementation should be read as computing *marginal* goal probabilities: each goal-specific simulation starts from the current total portfolio and does not deduct earlier goal spending before evaluating later goals. A full professional engine would estimate a joint, path-dependent success probability across retirement income, college outflows, and discretionary property spending.

4.5. Goal-Weighted Score

The notebook assigns priority weights to the three ranked goals: retirement security receives 60 percent, college funding receives 28 percent, and the vacation home receives 12 percent. Let ω_g denote the weight for goal g . The overall goal-attainment score is

$$G^{(h)} = \sum_g \omega_g P_g^{(h)}. \quad (10)$$

This equation encodes the client's value hierarchy into the objective function. A strategy that improves the vacation-home probability at the expense of retirement security is therefore penalized, even if it raises total expected wealth. The score should not be interpreted as expected utility, a certainty equivalent, or a comprehensive welfare measure. It is a transparent triage metric that forces the search process to respect the client's stated ordering: protect retirement first, improve college funding second, and treat the vacation home as discretionary.

4.6. Tax-Efficiency Score and Risk Flags

The product menu assigns tax-efficiency characteristics to each investment vehicle. The notebook computes a simplified tax-efficiency score as a weighted average of product tax-efficiency indicators:

$$TE^{(h)} = \sum_j q_j^{(h)} e_j, \quad (11)$$

where $e_j = 1.0$ for tax-efficient products and $e_j = 0.4$ for products not marked tax efficient. The simulator also generates risk flags for drawdown excess and cash allocations above 20 percent. It contains a prototype illiquidity check for a `private_equity` allocation key, while the notebook’s actual illiquid-product controls are primarily handled through the generation prompt and LLM suitability review because `private_equity` appears in `product_mix`, not in the allowed asset-class glossary. These flags do not automatically reject a strategy and are stored on the `SimResult` as audit diagnostics; a production version should surface them directly in the advisor-review and evaluation-agent prompts.

4.7. Shadow-Portfolio Proxy

After selecting the best admissible strategy in a cycle, the notebook computes a short-horizon shadow-portfolio proxy. It runs 500 simulated 21-trading-day paths from a unit starting value, using daily normal return increments derived from the annual asset-class return and volatility assumptions. The reported value is the median simulated 30-day return. This proxy is not a full paper-trading system or implementation recommendation; it is a lightweight monitoring artifact shown in the cycle summary, advisor-review prompt, and final dashboard.

4.8. Governance-Constrained Selection

Within a cycle, the provisional best strategy is selected by goal-attainment score after excluding strategies with an `UNSUITABLE` verdict. Strategies marked `BORDERLINE` are still simulated but remain visible to advisor review and outer-governance checks:

$$h^* = \arg \max_{h \in \mathcal{H}_{\text{adm}}} G^{(h)}, \quad \mathcal{H}_{\text{adm}} = \{h : \text{verdict}(h) \neq \text{UNSUITABLE}\}. \quad (12)$$

The deterministic heuristic passed into the evaluation-agent prompt is stricter:

$$\begin{aligned} \text{StopSuccess}(h^*) = \mathbf{1} [& G^{(h^*)} \geq 0.65 \\ & \wedge G^{(h^*)} - G^{(0)} \geq 0.05 \\ & \wedge \neg \text{Fragile}(h^*) \\ & \wedge \text{AdvisorApproved}(h^*)], \end{aligned} \quad (13)$$

where $G^{(0)}$ is the baseline goal-attainment score. The `AdvisorApproved` condition includes `APPROVE` and `APPROVE_WITH_MODS`. This formulation captures the heuristic logic of the notebook. The final decision is returned by a constrained LLM call that is given this heuristic and the same rules; the code additionally converts an attempted `CONTINUE` at the third round into `STOP_MAX_ROUNDS`. A production implementation should enforce the full stop-success predicate after the LLM response.

5. The WealthEvaluationAgent Decision Protocol

The `WealthEvaluationAgent` formalizes the continuation problem. In traditional wealth planning, the decision to keep researching, revise a plan, or recommend a strategy is often made through advisor judgment supported by planning software and client conversation. The notebook does not attempt to replace that judgment. Instead, it makes the decision protocol explicit and auditable.

5.1. Motivation

The motivation for a separate evaluation agent is governance discipline. The component that generates strategies should not be the same component that decides whether the process is complete. The strategy generator is designed to explore the planning space. The evaluation agent is designed to control that exploration. This separation mirrors the professional distinction between analysis, recommendation, advisor review, and client counseling.

The agent applies four broad criteria. First, the strategy must be suitable. Second, it must be materially better than the baseline. Third, it must be sufficiently robust under stress scenarios. Fourth, it must survive advisor review. A strategy that fails any of these conditions should generally trigger another cycle or be rejected.

5.2. Two-Step Protocol

The decision protocol has two stages. The first is a deterministic heuristic. If the advisor review is `REJECT`, the agent continues. If the advisor approves or approves with modifications, the goal-attainment score is at least 65 percent, improvement over baseline is at least five percentage points, and the strategy is not fragile, the heuristic recommends `STOP_SUCCESS`. Otherwise, it recommends `CONTINUE`.

The second stage is a constrained LLM decision. The evaluation prompt receives the round number, maximum rounds, strategy name, goal score, baseline score, improvement, fragility status, CFP verdict, CFP rationale, required modifications, and heuristic recommendation. It must return a structured decision object:

```
{
  "decision": "CONTINUE | STOP_SUCCESS | STOP_MAX_ROUNDS",
  "reasoning": "short explanation grounded in metrics and CFP review",
  "failure_summary": "what to improve next cycle"
}
```

The schema is intentionally narrow. The LLM may reason about the evidence, but it may not alter the contract, expand the decision vocabulary, or conceal the failure mode. If the maximum round boundary has been reached, the code prevents the model from continuing indefinitely by converting an attempted `CONTINUE` decision into `STOP_MAX_ROUNDS`.

5.3. Failure Context

A key feature of the protocol is failure-context accumulation. The current code records the best strategy from each continuing round, the goal score, CFP verdict, required modifications, AI diagnosis, and seed names for the next cycle. That failure context is injected into the next `WealthStrategyLoop`, instructing the LLM not to regenerate failed strategies and to propose better-suited alternatives.

In an AI4S framework, failure is not discarded. It becomes an experimental result. A strategy that fails because it is unsuitable, fragile, too expensive, behaviorally unrealistic, or misaligned with goal hierarchy teaches the system where not to search next. The `WealthEvaluationAgent` is the mechanism that converts failed wealth strategies into structured memory.

6. Suitability, Behavioral, and Governance Safeguards

AI-assisted wealth-planning systems require safeguards because financial advice is a high-stakes professional domain. An erroneous recommendation can damage a client’s retirement security, create unnecessary tax costs, increase behavioral failure risk, violate fiduciary duties, or expose the advisor to professional liability. In seasoned advisory practice, a recommendation must be suitable not only for stated risk tolerance but also for risk capacity, liquidity reserves, time horizon, tax status, product cost, account type, complexity, and the client’s ability to stay invested. The notebook therefore embeds safeguards at several levels: suitability review, structured strategy representation, code-controlled simulation, stress testing, behavioral review, advisor approval, and stopping logic.

6.1. Suitability Gate

The suitability gate is mandatory. Claude is instructed to act as a compliance officer applying fiduciary suitability rules. Each generated strategy receives a verdict: `SUITABLE`, `BORDERLINE`, or `UNSUITABLE`. Strategies classified as `UNSUITABLE` are excluded from simulation ranking.

This is a critical design choice. The system does not run the Monte Carlo simulator for an unsuitable strategy or present its simulated goal probability as a legitimate option. In professional domains, quantification can create false legitimacy. The suitability gate prevents a strategy from gaining analytical authority before it has cleared the minimum fiduciary threshold.

6.2. Structured Strategy Safeguards

The strategy-generation prompt requires structured JSON output. Each strategy must include a name, two-sentence description, allocation dictionary, product mix, annual contribution, account priority list, concise rationale, target goal rank, and estimated risk level. This structure converts an open-ended planning narrative into an inspectable object.

Structured representation reduces ambiguity. A strategy that says “improve diversification” is not directly testable. A strategy that specifies asset class weights, product mix, annual contribution, and account priorities is testable. The system therefore forces financial language into a computational form before simulation.

6.3. Numerical Safeguards

The code-controlled simulation engine protects against hallucinated outcomes. It computes portfolio return, volatility, Sharpe ratio, fee drag, terminal wealth distribution, goal probabilities, goal-weighted score, tax-efficiency score, drawdown, and risk flags from encoded assumptions and strategy inputs. This decomposition makes the result inspectable.

A production version would require additional controls, including validated capital-market assumptions, historical scenario analysis, tax-lot modeling, account-level optimization, retirement-income modeling, insurance integration, estate planning, sequence-risk analysis, withdrawal policy, and independent model validation. The notebook demonstrates the architecture rather than a production financial plan.

6.4. Behavioral-Review Safeguards

The `AdvisorReviewAgent` models professional skepticism and behavioral awareness. Robert Okafor reviews not only performance metrics but also whether the plan respects the client’s ranked goals, whether the clients are likely to stick with the allocation, whether fees are justified, and what emotional risks should be disclosed. His output includes behavioral warnings and re-plan triggers.

This is essential because the best plan is not necessarily the one that maximizes a mathematical objective. It is the one the client can understand, accept, fund, and maintain during stress. A strategy that causes panic selling during the first major drawdown is not actually a good strategy, even if its expected value is attractive. The advisor layer therefore serves as a behavioral commitment check: it asks whether the plan can plausibly survive the client's next bad market experience, not only whether it performs well in a spreadsheet.

6.5. Stopping Safeguards

The maximum number of rounds prevents uncontrolled search. The minimum goal-score threshold prevents the system from stopping on a weak plan. The minimum improvement threshold prevents the system from recommending changes that do not materially improve the baseline. The stress-fragility rule prevents a strategy from advancing if it collapses under adverse conditions. The advisor-review condition prevents model confidence from replacing professional accountability. Together, these controls convert AI-assisted wealth planning into governed strategy exploration.

7. LLM Integration

The LLM is integrated as a constrained reasoning and generation component. It is not treated as a fiduciary, a portfolio optimizer, a regulator, or the final decision maker. Its primary functions are strategy generation, suitability screening, strategic feedback, next-cycle seed generation, CFP-style review, and evaluation-agent reasoning. Each function is governed by a prompt contract and expects structured output.

The notebook uses Anthropic Claude through a centralized function. The model identifier is stored in MODEL:

```
MODEL = 'claude-haiku-4-5-20251001'
```

The wrapper function is:

```
ask_claude(system, user, max_tokens=1200)
```

This function sends system and user prompts to the model and returns text. The setup installs the Anthropic SDK, retrieves the API key from Colab userdata, and constructs an Anthropic client. A separate parsing utility removes common Markdown code fences and parses JSON. This design is important because LLM outputs can be semantically rich but structurally unstable. The surrounding code enforces the interface.

The implementation uses six prompt families:

1. **Strategy generation.** Claude proposes exactly five investment strategies, each with allocation, product mix, annual contribution, account priorities, target goal, rationale, and risk label.
2. **Suitability review.** Claude acts as compliance officer and assigns `SUITABLE`, `BORDERLINE`, or `UNSUITABLE` verdicts.
3. **Strategic feedback.** Claude evaluates the selected strategy after simulation and stress testing, identifying the largest remaining risk and one concrete improvement.
4. **Next-cycle seed generation.** Claude proposes two refined strategic directions based on feedback and rejected strategy names.
5. **Advisor review.** Claude simulates Robert Okafor, CFP, and returns a professional verdict, modifications, client talking points, behavioral warnings, and re-plan triggers.

6. **Evaluation-agent decision.** Claude helps the outer governor decide whether to continue, stop successfully, or halt at maximum rounds.

The most important design principle is that LLM judgment is never allowed to replace fiduciary governance. The LLM may propose strategies and produce structured commentary, but it does not compute outcomes independently, does not bypass the suitability gate, does not override the goal hierarchy, and does not replace advisor review. Stronger models may improve the quality of generated strategies, but they do not remove the need for suitability screening, code-controlled simulation, stress testing, advisor approval, and audit logs.

8. Stress Testing Scenarios

Stress testing asks whether a candidate wealth strategy remains credible when important assumptions deteriorate. A strategy that looks favorable under the base case may become weak if markets crash, inflation rises, or poor returns occur early in the planning horizon. The notebook therefore subjects the best strategy in each cycle to three coded stress proxies.

The scenarios are illustrative rather than exhaustive. Their purpose is to model the governance principle that a recommendation should not be judged only in the base case. A production system would include historical stress periods, tax shocks, employment loss, health shocks, insurance events, divorce, college cost inflation, liquidity demands, and retirement-spending uncertainty.

Scenario A — Market Crash. The notebook approximates a major risk-asset drawdown by rerunning the candidate allocation and applying a 22 percent terminal-wealth haircut to the median simulated result. The code includes crash-adjusted return and volatility calculations, but those quantities are not used in the reported stress value; the dashboard and fragility check use the haircut-adjusted median terminal wealth. This scenario tests whether the strategy remains viable if the client experiences a large market shock before the plan has had time to compound.

Scenario B — Inflation Spike. The system approximates an inflation shock by rerunning the candidate allocation and applying a 12 percent terminal-wealth haircut, described in the notebook as a two-percentage-point real-return drag over the fifteen-year horizon. This scenario tests whether the strategy remains credible when purchasing-power assumptions deteriorate, which is especially important for retirement security and college funding.

Scenario C — Sequence-of>Returns Risk. The system uses the 15th percentile of a fresh fifteen-year terminal-wealth simulation as a proxy for sequence-of-returns pressure. This scenario is material because clients do not experience average returns. The order of returns matters, especially when future contributions, goal timing, and retirement horizons are involved, even though the notebook's current proxy is not a full year-by-year adverse-return ordering model.

A strategy is classified as fragile if the minimum stressed terminal value falls below 70 percent of the retirement target. Fragility does not mean that the strategy has no value. It means that the strategy should not proceed without modification, additional funding, lower goal expectations, increased defensive allocation, or explicit client discussion. In the AI4S loop, fragility becomes structured feedback for the next cycle.

9. Investment Committee Readiness

A top-tier investment committee would not approve this system because an LLM can produce plausible recommendations or because a simulated goal score improves. The appropriate committee question is more demanding: whether the workflow creates a controlled, reviewable, and challengeable process for exploring

household wealth strategies before any client-facing recommendation is made. On that standard, the notebook should be evaluated as a governed research prototype, not as an approved advisory platform.

The committee review should separate three approval objects. The first is the *methodology*: whether the architecture cleanly separates strategy generation, suitability screening, simulation, advisor review, and final governance. The second is the *model*: whether assumptions, code, prompts, stochastic outputs, and decision thresholds are versioned, validated, and reproducible. The third is the *client recommendation*: whether a specific strategy is suitable, implementable, tax-aware, behaviorally durable, and documented under the firm’s fiduciary and compliance obligations. The notebook is strongest on the first object, partial on the second, and explicitly insufficient for the third without human fiduciary review and production-grade controls.

Table 2 summarizes the committee-level challenge process.

Table 2: Investment-committee challenge framework for the prototype.

Committee question	Notebook evidence	Production control required
Client objective	Ranked goals, explicit targets, flexibility labels, and priority weights.	Signed discovery record, IPS alignment, household cash-flow inventory, and documented goal hierarchy.
Assumption governance	Synthetic returns, volatilities, correlations, inflation, taxes, and safe-withdrawal rate.	Committee-approved capital-market assumptions, version history, sensitivity analysis, and independent validation.
Model risk	Code computes portfolio statistics, Monte Carlo paths, goal scores, stress proxies, and stop heuristics.	Model inventory entry, owner, validation report, change controls, benchmark tests, and calculation audit trail.
Suitability and fiduciary review	LLM suitability gate plus simulated Robert Okafor, CFP review.	Deterministic compliance rules, licensed-advisor approval, conflict review, best-interest rationale, and supervisory sign-off.
Implementation feasibility	Strategy JSON includes allocation, product mix, contributions, account priorities, and rationale.	Account-level constraints, tax-lot analysis, product due diligence, trading workflow, fee schedule, and client consent.
Monitoring and escalation	Stress tests, shadow return proxy, behavioral warnings, re-plan triggers, and decision log.	Formal review calendar, drift thresholds, exception reporting, client-contact triggers, and remediation playbook.

This framing also clarifies what the committee should challenge. First, the goal score is useful for search but is not a client-ready probability of success; it uses marginal goal probabilities and simplified accumulation dynamics. Second, the market universe is synthetic and should not be confused with a house view or approved capital-market assumption set. Third, the suitability gate is partly prompt-based and must not be treated as a deterministic compliance engine. Fourth, fee drag, tax efficiency, and account priorities are reported and reviewed, but not yet integrated into a full tax-aware implementation optimizer. Fifth, stress testing is illustrative and should be expanded before production use to include historical drawdowns, interest-rate shocks, tax-law changes, job-loss or income-shock scenarios, college-cost inflation, health events, and retirement-spending shocks.

An investment committee could defensibly approve the architecture for controlled research, analyst training, or supervised idea generation if seven conditions are met. The system must maintain immutable prompt, response, model-version, code, and assumption logs; use only approved client data and privacy controls; require human fiduciary approval before client communication; prohibit the LLM from changing assumptions or approving its own output; disclose the synthetic nature of the capital-market universe; escalate all borderline, fragile, illiquid, complex, or high-fee strategies; and document why each continuing or stopping

decision is consistent with the client’s ranked goals. Those conditions convert the prototype from a persuasive AI demonstration into an institutionally reviewable decision-support workflow.

The committee should not approve automated deployment from the notebook as written. The correct approval boundary is narrower: the notebook demonstrates a credible governance architecture that can be advanced to model validation, compliance design, and advisor-workflow integration. It is strongest where it makes judgment explicit and most incomplete where real advisory systems are hardest: data quality, tax lots, account location, withdrawal sequencing, product due diligence, jurisdiction-specific fiduciary rules, privacy, and supervisory controls. A rigorous investment committee would insist on these limitations because they are not defects in wording; they are the boundary between a research prototype and advice that can be delivered to a household.

10. Discussion and Limitations

The architecture should be evaluated as a governance design, not as a claim of real-world financial-planning completeness. Its primary strength is the explicit connection between generative AI, suitability screening, stochastic simulation, goal hierarchy, stress analysis, behavioral review, advisor judgment, decision logging, and investment-committee challenge. Its primary limitation is that the market, regulatory, tax, and behavioral environments are synthetic and simplified.

10.1. Contributions

The first contribution is a formal mapping from AI4S to household wealth planning. The paper shows how observation, representation, strategy generation, suitability screening, simulation, stress testing, feedback, and revision can be implemented as a professional advisory workflow. The second contribution is the separation between `WealthStrategyLoop` and `WealthEvaluationAgent`. This separation prevents the component that generates strategies from also owning the final governance decision. The third contribution is the explicit advisor-in-the-loop layer represented by `AdvisorReviewAgent`. The fourth contribution is the insistence that goal probabilities be computed through seeded, code-controlled simulation rather than accepted as LLM assertions. The fifth contribution is the integration of behavioral warnings and re-plan triggers, recognizing that financial advice is not only quantitative but also behavioral and relational. The sixth contribution is an explicit investment-committee challenge framework that separates approval of the methodology, the model, and any eventual client recommendation.

10.2. Limitations

The most important limitation is that the market universe is synthetic. Real wealth planning would require validated capital-market assumptions, actual client account data, tax-lot information, household balance sheet detail, insurance coverage, estate-planning documents, Social Security assumptions, retirement spending estimates, pension data, debt schedules, and jurisdiction-specific tax rules.

A second limitation is that the advisor is simulated by an LLM. This is useful for prototyping, but it is not a substitute for licensed professional review. In a real advisory practice, the system should escalate candidate strategies to a fiduciary professional before any client communication or implementation.

A third limitation is stochasticity. LLM-generated strategies may vary across runs. Diversity can improve search, but it also requires logging, versioning, reproducibility controls, and prompt governance. A production system would need to store prompts, responses, model versions, market-assumption versions, client-data versions, and calculation hashes.

A fourth limitation is that the simulation is simplified. Real financial plans are multi-period, tax-sensitive, account-aware, and behaviorally contingent. They involve rebalancing rules, withdrawal policies, tax location, Roth conversion analysis, charitable planning, insurance, estate planning, education-specific rules, and changes in income, expenses, and goals over time. In particular, the notebook records account limits and income limits but does not deterministically enforce contribution eligibility, tax location, Roth IRA income constraints, tax-lot realization, or goal-specific withdrawals. The goal probabilities are therefore useful comparative diagnostics, not a complete client-ready probability-of-success report. The notebook demonstrates the architecture, not a complete advisory platform.

10.3. Future Work

Future work should proceed along five lines. First, the synthetic market universe should be replaced with validated, versioned capital-market assumptions and historical scenario libraries. Second, the simulation engine should become account-aware, tax-aware, and multi-period, incorporating rebalancing, withdrawals, tax lots, contribution limits, Social Security, retirement-income spending rules, and goal-specific cash-flow waterfalls. Third, the suitability gate should be strengthened with formal compliance rules, jurisdiction-specific fiduciary obligations, investment-policy-statement checks, and documented best-interest rationales. Fourth, the advisor-review layer should be connected to actual professional workflows, including advisor notes, client discovery records, IPS documents, proposal review, implementation checklists, and annual review triggers. Fifth, the dashboard should export model cards, risk registers, client memos, reproducibility manifests, behavioral-risk summaries, and audit bundles.

A particularly important extension is client-centered governance. Wealth planning is not merely the maximization of goal probability. Clients may care about confidence, simplicity, control, liquidity, taxes, family obligations, legacy, privacy, and emotional comfort. A mature AI4S wealth-planning system should incorporate those objectives explicitly.

11. Conclusion

This paper has presented an AI4S architecture for agent-governed financial advice. The system combines a synthetic market universe, a structured client profile, ranked goals, a Monte Carlo simulation engine, LLM-generated wealth strategy search, mandatory suitability screening, stress testing, shadow-portfolio monitoring, fiduciary advisor review, behavioral warnings, and a supervisory evaluation agent. Its central design principle is that AI creativity must be embedded inside professional wealth-management governance.

The architecture is not a financial-advice chatbot. It is a controlled discovery loop. The LLM proposes strategies, but does not authorize them. The suitability gate excludes strategies that should not be simulated as legitimate recommendations. The Monte Carlo engine computes outcome distributions and goal probabilities. The goal hierarchy ensures that retirement security, college funding, and vacation-home aspirations are not treated as interchangeable. The stress tests expose fragility under market crash, inflation, and sequence-of-returns scenarios. The advisor review introduces professional skepticism and behavioral realism. The evaluation agent coordinates these layers and records why the process continues, stops successfully, or halts at the maximum-round boundary. The investment-committee layer then asks the institutional question: what can be approved for research, what must go through model validation, and what still requires human fiduciary approval before reaching a client.

The deeper point is that wealth planning has always required a dialogue between quantitative analysis and human judgment. The quantitative side produces returns, volatilities, Sharpe ratios, terminal wealth distributions, funded status diagnostics, and goal probabilities. The human side interprets risk capacity, risk tolerance, emotional resilience, family priorities, liquidity needs, tax tradeoffs, account constraints, and

whether the client will actually follow the plan. The AI4S loop does not eliminate that tension. It sharpens both sides simultaneously.

The companion notebook makes this claim operational. It shows how one household planning problem can be converted into a structured AI4S loop: propose, screen, simulate, stress-test, advise, decide, and iterate. The system cannot replace the judgment that comes from knowing the client, understanding their family, and helping them behave well during uncertainty. Those judgments belong to the human advisor. What the system can do is organize the search space, evaluate alternatives in parallel, preserve failure context, and make recommendations more transparent. Its value lies not in replacing financial judgment, but in making financial judgment more informed, disciplined, auditable, and professionally defensible.

Appendix A. Companion Computational Notebook

The paper is accompanied by one computational notebook:

[Open the companion Colab notebook](#)

The notebook implements the architecture described in this paper. It contains a complete, self-contained demonstration of a governed agentic wealth-planning workflow. The executable implementation is organized into nine code cells, accompanied by contextual, architecture, reference, and conclusion Markdown:

1. Setup, dependencies, Claude client, and parsing utilities.
2. Synthetic market universe: asset classes, products, account types, fees, regulations, inflation, and tax assumptions.
3. Client profile and ranked goals for Sam and Taylor Reeves.
4. Monte Carlo engine and goal-attainment simulator.
5. `WealthStrategyLoop`.
6. `AdvisorReviewAgent`.
7. `WealthEvaluationAgent`.
8. Orchestrator and execution entry point.
9. Final wealth-plan dashboard, saved dashboard figure, and client memo.

The notebook should be interpreted as a methodological prototype. It does not represent investment advice, does not recommend any security, and does not create an advisor-client relationship. Its purpose is to show how AI-assisted wealth planning can be governed through explicit client facts, synthetic market assumptions, structured agent outputs, suitability gates, Monte Carlo simulation, stress tests, advisor review, behavioral warnings, and decision logs.

Appendix B. Workflow Diagram

Figure B.1 provides the workflow diagram corresponding to the agent-governed AI4S financial-advice architecture described in Section 2. It summarizes the movement from client facts and a synthetic market universe to strategy generation, suitability screening, Monte Carlo simulation, stress testing, advisor review, and evaluation-agent decision.

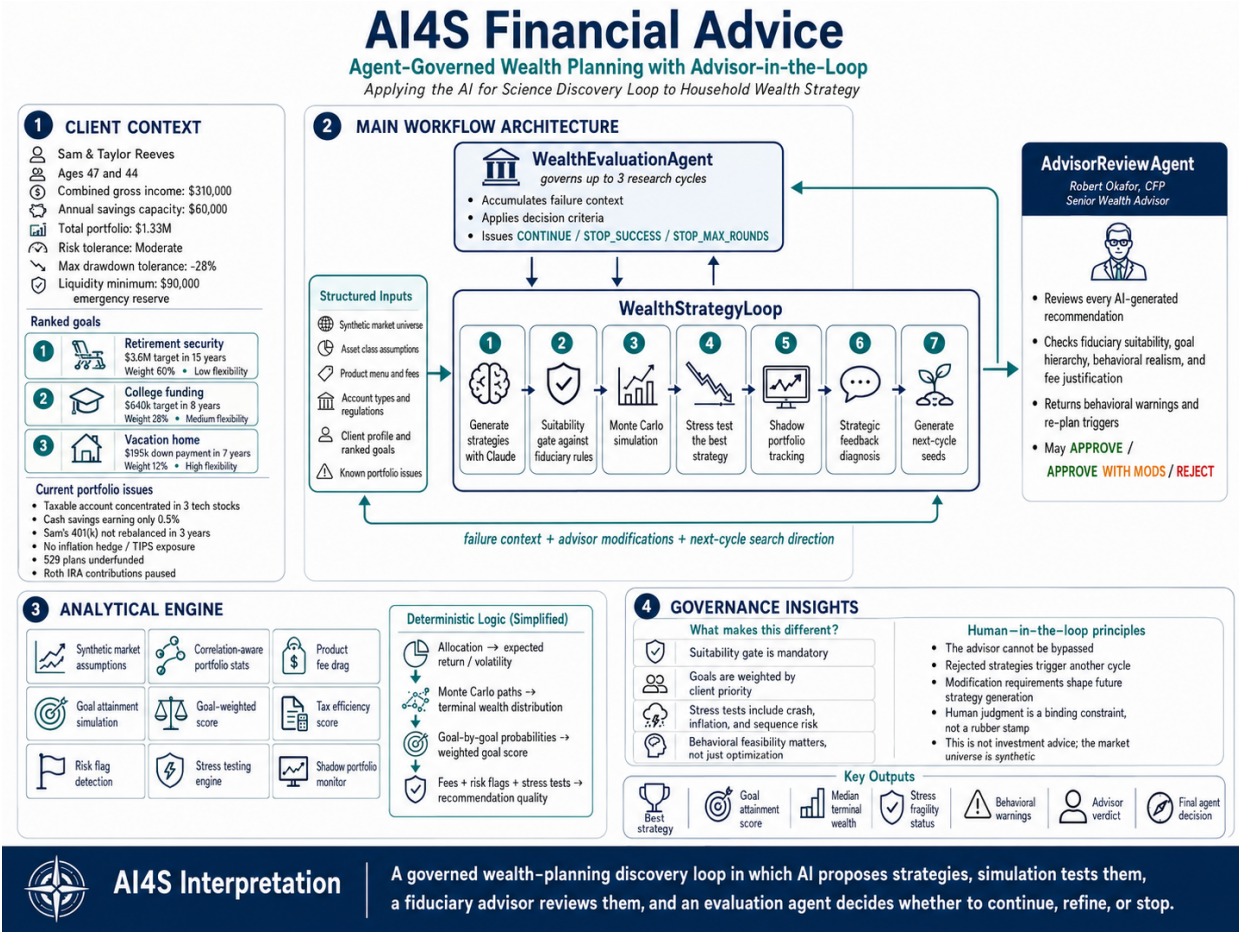


Figure B.1: Workflow diagram for the agent-governed AI4S financial-advice architecture.

References

- [1] H. Wang *et al.*, “Scientific discovery in the age of artificial intelligence,” *Nature*, vol. 620, pp. 47–60, 2023.
- [2] J. Jumper *et al.*, “Highly accurate protein structure prediction with AlphaFold,” *Nature*, vol. 596, pp. 583–589, 2021.
- [3] B. Burger *et al.*, “A mobile robotic chemist,” *Nature*, vol. 583, pp. 237–241, 2020.
- [4] H. Markowitz, “Portfolio selection,” *The Journal of Finance*, vol. 7, no. 1, pp. 77–91, 1952.
- [5] W. F. Sharpe, “Capital asset prices: A theory of market equilibrium under conditions of risk,” *The Journal of Finance*, vol. 19, no. 3, pp. 425–442, 1964.
- [6] J. L. P. Brunel, *Goals-Based Wealth Management: An Integrated and Practical Approach to Changing the Structure of Wealth Advisory Practices*. Wiley, 2015.
- [7] D. Kahneman and A. Tversky, “Prospect theory: An analysis of decision under risk,” *Econometrica*, vol. 47, no. 2, pp. 263–291, 1979.
- [8] National Institute of Standards and Technology, *Artificial Intelligence Risk Management Framework*. NIST AI 100-1, 2023.
- [9] U.S. Securities and Exchange Commission, “Regulation Best Interest: The Broker-Dealer Standard of Conduct,” Release No. 34-86031, 2019.